

GenAI Engineer — Technical AssessmentObjective

Build an AI-powered Knowledge Assistant that can answer questions from a provided set of documents.

The goal is to demonstrate your understanding of LLM applications, RAG, embeddings, retrieval, prompt engineering, backend development, and evaluation.

You are free to choose the technology stack, LLM provider, embedding model, and vector database.

Problem Statement

You will be provided with a collection of documents containing information about a fictional company/product.

Build a chatbot that allows users to ask questions about these documents.

The system should retrieve relevant information from the knowledge base and generate an accurate answer based on the retrieved context.

The assistant should provide citations/sources for its answers wherever possible.

Requirements1. Document Ingestion

Build a pipeline that:

- Loads the provided documents
- Extracts their content
- Splits the content into appropriate chunks
- Generates embeddings
- Stores the embeddings in a searchable data store/vector database

Explain the reasoning behind your chunking strategy.

2. Retrieval

Implement a retrieval mechanism that identifies the most relevant information for a user's question.

The system should:

- Retrieve relevant chunks
- Provide appropriate context to the LLM
- Avoid unnecessarily large context windows

- Handle cases where relevant information cannot be found

3. LLM Generation

Use an LLM to generate answers based on the retrieved context.

The assistant should:

- Answer questions using the provided knowledge base
- Avoid making up information
- Clearly indicate when the information is not available
- Handle follow-up questions
- Produce clear and concise responses

4. Citations

Answers should include references to the source documents used to generate the response.

For example:

Answer:

"The refund period is 30 days."

Sources:

- refund_policy.pdf — Page 4

5. Backend API

Expose the chatbot through an API.

Example:

POST /chat

Request:

```
{  
  "question": "What is the refund policy?"  
}
```

Response:

```
{  
  "answer": "The refund period is 30 days.",  
  "sources": [  
    {  
      "document": "refund_policy.pdf",  
      "page": 4  
    }  
  ]  
}
```

The exact API design is up to you.

6. User Interface

Create a simple interface through which a user can:

- Ask questions
- View answers
- View sources/citations
- Continue a conversation

The UI does not need to be highly polished.

Evaluation

Your solution will be evaluated on:

RAG Quality

- Retrieval accuracy
- Relevance of retrieved context
- Ability to answer questions correctly
- Hallucination handling
- Source attribution

LLM Implementation

- Prompt design
- Context management
- Model selection
- Structured outputs where appropriate
- Handling of edge cases

Engineering

- Code quality

- Architecture
- API design
- Error handling
- Maintainability
- Testing

AI System Design

- Understanding of RAG architecture
- Embedding strategy
- Chunking strategy
- Retrieval strategy
- Scalability considerations
- Cost considerations

Evaluation Questions

Your implementation should perform reasonably well on the following types of questions:

- A question whose answer exists clearly in one document.
- A question whose answer requires information from multiple documents.
- A question requiring reasoning over retrieved information.
- A question whose answer does not exist in the knowledge base.
- A question containing misleading or incorrect assumptions.

The system should not confidently invent an answer when the required information is unavailable.

Bonus

The following are optional but will be considered positively:

- Hybrid search

- Reranking
- Query rewriting
- Conversational memory
- Retrieval evaluation
- LLM evaluation
- Streaming responses
- Structured outputs
- Guardrails
- Observability/logging
- Caching
- Cost optimisation
- Prompt/version management
- Dockerisation
- Automated tests
- Evaluation dataset
- Confidence/relevance scoring

AI Tools

AI tools are allowed and encouraged.

You may use tools such as ChatGPT, Claude, Cursor, GitHub Copilot, or other AI coding assistants.

However, you should be able to explain the implementation and architectural decisions made in your solution.

During the next round, we may ask you to modify, debug, or explain parts of your implementation.

Documentation

Include a README containing:

- Architecture overview

- Technology choices
- Setup instructions
- How to run the application
- How to run tests
- RAG pipeline explanation
- Chunking strategy
- Embedding/model choices
- Retrieval approach
- Prompting approach
- Assumptions
- Known limitations
- What you would improve with additional time

Time:

Expected effort: 4–5 hours.

Focus on demonstrating strong engineering and GenAI fundamentals rather than building an unnecessarily large application.

Good luck!